We're using DateTimeImmutable to give us the same date and time for each demonstration. Each of the calls to setDate(), setTime(), etc., returns a DateTime reference or a new DateTimeImmutable instance, which means we can take advantage of a technique called *method chaining* and make multiple calls one after the other. Each example should be self-explanatory: setDate() (to the third day of the ninth month) and setISODate() (to the second day of the thirty-sixth week). Change the date accordingly (keeping the same time and time zone), setTime() (fifth hour, zeroth minute). Adjust only the time; setTimezone() requires a DateTimeZone instance and does exactly what it says, as does setTimestamp().

Bear in mind that setTimestamp() is always UTC (GMT+00:00), and also be aware that the order in which setTimezone() is called relative to setTime() et al. will affect the final outcome:

```
$dti = new DateTimeImmutable('@1725148800');
$new1 = $dti->setTimeZone(new DateTimeZone('PST'))
    ->setISODate(2024, 36, 2)
    ->setTime(5, 0);

$new2 = $dti->setISODate(2024, 36, 2)
    ->setTime(5, 0)
    ->setTimeZone(new DateTimeZone('PST'));

print $new1->format(DateTime::COOKIE) . PHP_EOL;
print $new2->format(DateTime::COOKIE);

Output:
Tuesday, 03-Sep-2024 05:00:00 PST
Monday, 02-Sep-2024 21:00:00 PST
```

We'll look at DateTime::add() next, together with its required argument type: DateInterval.

#### DateInterval, DatePeriod

The final two classes in the date extension are used for measuring and iterating time periods. A DateInterval object represents the amount of time between one DateTime and another. DatePeriod allows you to iterate in fixed time periods from a starting time. DateInterval objects are constructed using a string that is a formatted amount of time: these strings must start with the letter P (for period) followed by integers and the following designator letters: